

Hackathon Submission

Grade: A

Status: Pass (automatic by LLM)

Feedback

High-level assessment

- This is a very polished project that convincingly simulates a real vacation-rental business. The landing page and widget feel production-grade, and the agent is thoughtfully constrained by deterministic tools and a clear system prompt.
- The end-to-end model is strong: a static storefront + BFF proxy + agent + custom tools. The proxy correctly hides credentials and streams SSE. The agent prompt and tools enforce "facts via tools only," curated suggestions, and price checks before quoting.
- Visuals/UX: The site is excellent. Accessibility, SEO, and microcopy are handled with care (skip link, alt text, semantic structure, JSON-LD, meta/OG). The floating chat, quick-view PDP, cookie consent, and graceful states elevate the demo.
- According to the spec PDF: the diagram on page 1 shows the Site -> Proxy -> Agent round-trip clearly, and the quality/metric bullets on page 2 match what the implementation aims to demonstrate.

What's great

- Guardrails
- Prompt-level rules are explicit (ask clarifying questions; 2-3 picks; never quote price without build_quote; availability gated by check_availability; capture_lead only on intent).
- Deterministic tools eliminate price hallucination; amenity matching is case-insensitive and requires ALL requested amenities (good).
- The widget renders tool activity (tool.started), giving transparency to users.
- Secure architecture
- JWT kept server-side; browser only hits /api/session and /api/chat.
- SSE proxied verbatim; 404 from upstream triggers client reset logic.
- TLS verify toggle for ALB hostnames is pragmatic and explained.
- Deployment ergonomics
- One-click local script and an AWS App Runner flow with an ECR bootstrap and access role creation.
- Pre-provisioning the worker after session creation to hide cold-start.
- Product finish
- Full branded landing with properties grid and PDP modal.
- Clear pricing copy and savings model (10% weekly + 5% loyalty).
- Images and content align with the collection; e.g., the team inspection photo, alpine chalet, Japanese machiya, and oceanfront villas mirror the assets listed in media_assets.json and shown throughout the page set.

Issues to fix before I can call this "production-ready"

1) Env var mismatch breaks cloud deploy

- server.py expects ADAL_BASE_URL; deploy_aws.sh sets ADAL_APP_URL. Result: the proxy will ignore your upstream URL in App Runner.

- Fix: in deploy_aws.sh, rename ADAL_APP_URL to ADAL_BASE_URL everywhere (and in the printed summary) or accept both in server.py.

2) Repo layout drift vs scripts

- run_local.sh cd's into proxy/ and tries to install -r requirements.txt there, and deploy_aws.sh builds with proxy/Dockerfile, but the submitted files place server.py and requirements.txt at repo root and do not include proxy/Dockerfile.
- Options:
 - Provide proxy/ with server.py, requirements.txt, and Dockerfile as referenced, or
 - Update run_local.sh and deploy_aws.sh to use the root paths and include a Dockerfile at the path you build from.
- 3) Missing Dockerfile (referenced but not present)
 - Please include proxy/Dockerfile (or a root Dockerfile) so the AWS deploy script works.
- 4) Requirements path
 - run_local.sh installs requirements.txt inside proxy/, but requirements.txt is at root. Either move the file or change the path.
- 5) Agent update strategy
 - _ensure_agent() reuses by name but never updates content if the prompt/tools change. Consider hashing prompt+tools and updating when the hash differs, or bumping the name/version automatically (Haven Concierge v2).
- 6) Availability tool always returns True
 - Fine for a demo; call this out more prominently in the chat copy when asked general availability. For a stronger demo, make availability deterministic by date range (e.g., block specific ranges).
- 7) Input validation and resilience
 - capture_lead trusts name/email. Consider minimal email validation and length caps server?side.
 - Add a simple POST /api/health to support container health checks.
 - Consider rate?limiting /api/session to avoid spam session creation.
- 8) Observability and QA
 - Add structured logs (session id, request id) and basic metrics (latency, tool-call counts).
 - Unit tests for tools (weekly/loyalty discount edges, amenity matching).
 - CI workflow (lint, test) to prevent regressions.

Smaller nits and suggestions

- Content Security Policy and stricter response headers in FastAPI (e.g., via starlette middleware).
- Static caching headers for site assets and fonts; images could use width/height to reduce CLS.
- The widget's markdown to HTML routine is intentionally minimal; sanitize or constrain if you later echo user content.
- Consider i18n and currency formatting based on locale in future work.
- Persist transcripts in localStorage to survive reloads (you even note this in Future work).

What I need from you to remedy the submission

- Either:
 - Provide the missing proxy/ directory with server.py, requirements.txt, and Dockerfile, OR
 - Update run_local.sh and deploy_aws.sh to reference the actual locations, and include the Dockerfile you build from.
- Fix the ADAL_APP_URL vs ADAL_BASE_URL mismatch in deploy_aws.sh or server.py.
- Include a quick "smoke test" section in README with exact cURL commands and expected JSON fragments for:

- POST /api/session -> returns { session_id }
- POST /api/chat/{id} with "Find an oceanfront villa in Hawaii under \$1300/night"
-> shows tool.started for search_properties and an assistant message that references HVN?MAUI?01.
- Optional but helpful: a temporary public App Runner URL and the JWT provisioning detail (or a recorded demo) so I can click through.

Guardrails and landing page rubric

- Guardrails: Strong. Prompt rules + deterministic tools + UI visibility of tool calls. Would be even stronger with code?level enforcement of "no price without build_quote" (e.g., tool?result citations surfaced in UI).
- Landing page: Excellent. It's not a harness; it's a believable storefront with thoughtful UX and SEO.

Overall verdict

- Impressive, clearly above the bar. The main blockers are deployment script mismatches and the absent Dockerfile; once fixed, this is showcase?quality.

****The following files could not be used for grading**** (unsupported format for ChatGPT): Part 1- Demoing haven_stays.mp4 (.mp4), Part 2- Demoing haven_stays.mp4 (.mp4), Haven_stays Spec.docx (.docx).

****The following files were not recognized**** (unknown format): agent.cpython-311.pyc (.pyc), Dockerfile (.dockerfile), server.cpython-311.pyc (.pyc), site.webmanifest (.webmanifest).

For grading, supported formats are: documents as PDF or plain text (.txt, .md, .rtf); images as .png, .jpg, .jpeg, .gif, .webp. Submit as a .zip or a single image.